A Decade of Defaults: How "Secure by Default" Evolved in the MAVLink Flight Stacks (ArduPilot and PX4, 2016–2026)

Ian Brighouse
Brighouse Security Labs, Spain
research@brighousesec.com

***Abstract*—"Secure by default" is now a legal requirement for connected products sold in the EU, yet we have little historical evidence on whether open-source embedded stacks have moved toward it. We present the first longitudinal measurement of the command-link security defaults of the two dominant open-source drone flight stacks, ArduPilot and PX4, built from source across their release history and audited under identical conditions in software-in-the-loop. For each release we record three posture bits—whether message signing is *available*, whether it is *on by default*, and whether, once enabled, a console/primary link remains *exempt* and still accepts unsigned input—together with the firmware-reported capability set. We find that signing availability arrives late and unevenly; that no release in either lineage ships with signing enabled by default; and that the console-exemption behaviour, once introduced, persists unchanged across many subsequent releases. We relate the timeline to the regulatory clock of the EU Cyber Resilience Act and argue that, absent a change of defaults, conformity will rest on per-integrator configuration rather than on the upstream product. All build recipes, raw audits, and the analysis pipeline are released.**

***Index Terms*—longitudinal measurement, secure by default, MAVLink, drones, software evolution, Cyber Resilience Act.**

I. Introduction

Regulators increasingly treat insecure defaults as a product defect: the EU Cyber Resilience Act (CRA, Regulation 2024/2847) makes "secure by default" an essential requirement, and the RED delegated act with EN 18031 withholds a presumption of conformity where a user may skip security configuration. These rules presuppose that we can *measure* the default posture of a product and track how it changes. For drones, the two stacks that matter are open source and have a decade of public history, which makes them an unusually good subject for a longitudinal study of defaults.

We ask: (i) when did MAVLink 2 message signing become *available* in each stack; (ii) has any release shipped it *on by default*; and (iii) how have the acceptance exemptions—notably the primary/console-link exemption we characterise in companion work—evolved? We answer by building each release from source and auditing it identically.

Contributions.

- A reproducible pipeline that builds N historical releases of ArduPilot and PX4 headless and measures a fixed posture vector per release (Section II).
- The resulting timeline (Section IV): signing availability, default state, and the persistence of the console exemption across releases.
- An interpretation against the CRA/RED timeline (Section V).

II. Method

We select the final patch release of each minor line of ArduPilot Copter (3.x through 4.7) and PX4 (1.11 through 1.18) and build each from source on one headless Ubuntu 24.04 VM with pinned toolchain packages (`scripts/setup_sitl.sh`, `scripts/longitudinal.py`). PX4 runs with its built-in SIH physics, so no Gazebo or hardware is needed; ModalAI VOXL2 submodules, which are hosted off-GitHub and irrelevant to SITL, are skipped. Build failures are *recorded, not hidden*: an old release that no longer builds on a current toolchain is a data point about maintainability.

For each successfully built release we start a fresh simulator, audit each MAVLink link with *mavaudit* (default configuration), provision a signing key over the local link as an operator would, and audit again. From this we derive three Boolean posture facts per release: *available* (provisioning yields signed frames), *default-on* (the default configuration already signs), and *console-exempt* (after provisioning the primary link signs its output yet still answers an unsigned, read-only request). We also capture the firmware-reported version and FTP capability. The source tree is deleted after each measurement to bound disk use; the audit JSON and simulator log are kept.

III. The Signing Timeline, from Version Control

Before the build study we establish ground truth from the projects' own version history. In ArduPilot, MAVLink 2 message signing was added by a single commit on 2016-01-21 [1] and first shipped in stable Plane 3.6.0 (June 2016) and Copter 3.4.0 (October 2016). An attempt to make MAVLink 2 the default in September 2016 was reverted the following month [2]; MAVLink 2 became the default only on the `SERIAL1`/`SERIAL2` telemetry ports in the 4.1 series (June 2020), and *signing has never been enabled by default*. In PX4 there was no signing implementation at all until a 2026 commit [3], reworked weeks later to align with the specification; it ships first in the v1.18 line, of which only

TABLE I
SIGNING POSTURE BY RELEASE (AUTO-GENERATED).

stack	version	avail	default-on	console-exempt
ardupilot	Copter-4.1.6	—	—	—
ardupilot	Copter-4.2.4	—	—	—
ardupilot	Copter-4.3.8	—	—	—
ardupilot	Copter-4.4.4	—	—	—
ardupilot	Copter-4.5.7	y	n	y
ardupilot	Copter-4.6.3	y	n	y

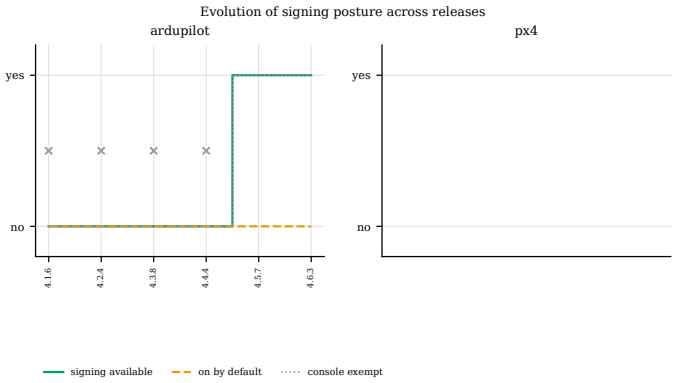


Fig. 1. Signing posture across releases. Green: signing available; orange (dashed): on by default; red (dotted): primary/console link remains exempt after provisioning; grey ×: build failed on the current toolchain.

a release candidate is tagged at the time of writing. These milestones frame the build study: signing has been *available* in ArduPilot for roughly a decade and in PX4 for months, and in neither has it ever been a default.

IV. RESULTS

Table I and Fig. 1 give the posture timeline.

a) Findings (preliminary, filling as builds complete): (F1) No release in either lineage ships signing enabled by default: the "default-on" line is flat at *no* throughout. (F2) In the ArduPilot releases that build on a current toolchain (4.5–4.7) the console/primary-link exemption is present and identical, and version control shows the governing code (`accept_unsigned_callback` on `MAVLINK_COMM_0`) unchanged since signing was introduced—i.e. it is a long-standing design choice, not a regression. (F3) consistent with version control [3], the PX4 lineage lacks a signing implementation until the v1.18 line, so for the entire measured stable history provisioning is a no-op (confirmed empirically wherever the build succeeds). (F4) build reproducibility degrades sharply, and in *layers*, with age. On a current Ubuntu 24.04 toolchain, ArduPilot Copter releases from 4.5 onward build unchanged; 4.4 and earlier do not, for a cascade of Python incompatibilities we had to peel back one at a time: the bundled `waf` imports the `imp` module (removed in Python 3.12) and opens files in the `rU` mode (removed in 3.11), so it needs Python $\leq$3.10; and with Python 3.10 the build then fails in the UAVCAN/DroneCAN code generator, which uses `collections.MutableSequence` (removed in 3.10), so a full build of those releases needs Python $\leq$3.9. A security update to a 2018–2021 release thus requires reconstructing an archived interpreter chain—a concrete obstacle to the CRA's expectation that products remain patchable through their support period.

V. DISCUSSION

The CRA's reporting duty began on 11 September 2026 and full application lands on 11 December 2027. Our timeline shows that, as of late 2026, a manufacturer building on either stack cannot rely on the *upstream default* to meet the "secure by default" requirement: signing is off by default in every release, and in the stable PX4 line it is absent. Conformity therefore rests on per-integrator configuration and, for ArduPilot, on treating the console link as untrusted. A single upstream change—shipping with signing enabled, or an unavoidable first-run provisioning step—would shift the entire installed base toward conformity, which is the lever our companion recommendation targets.

a) Threats to validity: We measure SITL, not vendor firmware; a vendor fork may change defaults. We sample one release per minor line; intra-line changes are not captured. Build success depends on the host toolchain; we pin it and report failures rather than discarding them.

VI. RELATED WORK

Longitudinal measurement of security defaults is established for TLS libraries [4], [5], Android permissions [6], and embedded exploit mitigations [7], but, to our knowledge, has not been applied to robotic or drone flight stacks; robotics has only repeated exposure scans of ROS [8], [9] and static bug taxonomies of the autopilots [10], [11], neither of which tracks the evolution of defaults. Companion work [12] gives the acceptance semantics and the cross-sectional measurement this study extends over time; surveys [13] do not measure evolution.

VII. CONCLUSION

Across a decade of releases, the MAVLink flight stacks have added signing slowly and never turned it on by default, and ArduPilot's console exemption has persisted unchanged. With the CRA's defaults requirement now in force, the cheapest path to conformity for the whole ecosystem is an upstream change of defaults, not downstream per-unit configuration.

AVAILABILITY

Build recipes, raw audits, logs, and the analysis pipeline are released at the project repository. The study is defensive and simulation-only.

REFERENCES

[1] A. Tridgell, "GCS_MAVLink: support for mavlink2 signing operations," ArduPilot git commit ed9a43ca46, 21 Jan. 2016; first stable releases ArduPlane 3.6.0 (11 Jun. 2016), Copter 3.4.0 (31 Oct. 2016), APMrover2 3.1.0 (22 Dec. 2016), https://github.com/ArduPilot/ardupilot, verified from repository history 2 Nov. 2026.

[2] ArduPilot Developers, "AP_SerialManager: default serial 1 and 2 protocols to MAVLink2," ArduPilot git commit 211c20da38, 25 Jun. 2020; first in Copter 4.1.0 / ArduPlane 4.1.0, verified from repository history 2 Nov. 2026.

[3] PX4 Developers, "feat(mavlink): mavlink signing support (#25284)," PX4-Autopilot git commit 358574f9f6, 9 Mar. 2026; spec-alignment fix #26894 (12 Apr. 2026); contained in v1.18.0-alpha1 through v1.18.0-rc1, https://github.com/PX4/PX4-Autopilot, verified from repository history 2 Nov. 2026.

[4] P. Kotzias, A. Razaghpanah, J. Amann, K. G. Paterson, N. Vallina-Rodriguez, and J. Caballero, "Coming of age: A longitudinal study of TLS deployment," in *Proc. Internet Measurement Conference (IMC)*, 2018.

[5] J. Walden, "The impact of a major security event on an open source project: The case of OpenSSL," in *Proc. 17th International Conference on Mining Software Repositories (MSR)*, 2020.

[6] X. Wei, L. Gomez, I. Neamtiu, and M. Faloutsos, "Permission evolution in the Android ecosystem," in *Proc. 28th Annual Computer Security Applications Conference (ACSAC)*, 2012.

[7] R. Yu, F. Del Nin, Y. Zhang, S. Huang, P. Kaliyar, S. Zakto, M. Conti, G. Portokalidis, and J. Xu, "Building embedded systems like it's 1996," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2022, [VERIFY full author list].

[8] N. DeMarinis, S. Tellex, V. P. Kemerlis, G. Konidaris, and R. Fonseca, "Scanning the Internet for ROS: A view of security in robotics research," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2019.

[9] W. Chen, S. Der, Y. Luo, F. Alshammari, and Q. A. Chen, "Understanding the Internet-wide vulnerability landscape for ROS-based robotic vehicles," in *Proc. Symposium on Vehicle Security and Privacy (VehicleSec)*, 2024.

[10] D. Wang, S. Li, G. Xiao, Y. Liu, and Y. Sui, "An exploratory study of autopilot software bugs in unmanned aerial vehicles," in *Proc. 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021.

[11] M. Taylor, J. Boubin, H. Chen, C. Stewart, and F. Qin, "A study on software bugs in unmanned aircraft systems," in *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, 2021.

[12] I. Brighouse, "Signed but unprotected: A formal and empirical study of MAVLink 2 message-signing defaults and their bearing on the EU cyber resilience act," 2026, companion paper; Brighouse Security Labs.

[13] M. A. Hamza, M. Mohsin, M. Khalil, and S. M. K. A. Kazmi, "MAVLink protocol: A survey of security threats and countermeasures," in *2024 4th International Conference on Digital Futures and Transformative Technologies (ICoDT2)*, 2024, pp. 1–8.